One Alphabet Reform, Six Spellings: How Uzbekistan's 2026 Latin Alphabet Affects LLM Tokenization, Accuracy and Training

Bilolbek Yoldoshxojayev
Presidential School in Namangan, Uzbekistan
bilolbek424@gmail.com

Abstract

In 2026 Uzbekistan adopted a revised Latin alphabet that replaces the digraphs o‘, g‘, sh and ch with the single letters ö, ğ, ş and ç. Uzbek is already typed with at least four different characters for the o‘/g‘ mark, and is still widely written in Cyrillic. We build a parallel version of FLORES-200 and Belebele in these spellings using a lexicon-aware converter, and measure tokenizer fertility, reading-comprehension accuracy on six open LLMs, API cost, and the effect of training on each spelling. Relative to how Uzbek is commonly typed, the new alphabet changes token counts by -3.0% to $+1.3\%$. Yet it lowers accuracy in every model, by 3.1–9.6 percentage points, while apostrophe variants have no significant effect. This looks like a transition cost: fine-tuning Gemma 3 4B on 4 million characters of new-alphabet text closes its gap (-6.8 to $+0.3$ points). A small model trained from scratch shows no consistent difference between old and new spelling in tokens, training time, energy or predictive quality.

1 Introduction

On 10 September 2026 the Senate of Uzbekistan approved a law amending the Latin-based Uzbek alphabet, and the President signed it on 21 October 2026 (Legal Alliance, 2026; Gazeta.uz, 2026). The reform replaces four letter combinations with single letters (o‘→ö, g‘→ğ, sh→ş, ch→ç), leaving 28 letters and one apostrophe; school textbooks are to be renewed in 2027–2031 (Legal Alliance, 2026). One stated motivation is digital: the o‘ mark is typed with several look-alike characters (U+02BB, ASCII ’, the backtick, U+2019), so identical words are treated as different by software (Legal Alliance, 2026).

Whether the reform helps language technology is publicly disputed. The director of the Institute of Uzbek Language, Literature and Folklore, Maqsud Asadov, has said that two-character letters cause technical obstacles for digital systems and AI, while the philologist Komil Jalilov argues that it makes no difference to AI whether a letter takes one symbol or two (Sobirov, 2026). Two recent reports measure token counts under the reform (Ro’zimatov, 2026; Ibadullaev et al., 2026), but we found no work that measures *model accuracy* or training on the new alphabet, or compares it against the apostrophe variants that already exist in real text.

We ask: **RQ1** How do the spellings change tokenizer fertility? **RQ2** How do they change LLM accuracy? **RQ3** What does this mean for API cost and usable context? **RQ4** Does normalizing to one form remove the gaps? **RQ5** Does training on the new alphabet differ in cost or outcome from training on the old one?

Our contributions are: (1) a parallel Uzbek version of FLORES-200 and Belebele, built with a lexicon-aware converter; (2) accuracy measurements of LLMs on the 2026 alphabet and on apostrophe variants, under three prompt frames; (3) a letter-level ablation separating ö/ğ from ş/ç; (4) a paired old-versus-new training comparison, for fine-tuning and from scratch.

2 Related Work

Tokenization and language variation. Tokenizers allocate unequal numbers of tokens across languages (Petrov et al., 2023), the tokenizer can matter as much as pretraining data for downstream performance (Rust et al., 2021), and less common spellings can be split into more tokens, with tokenizer choices affecting downstream tasks (Wegmann et al., 2025).

The 2026 reform. Ibadullaev et al. (2026) train SentencePiece and byte-level BPE tokenizers on Uzbek and find that the reform’s token savings come from tokenizers splitting the apostrophe off

the letter; with script splitting disabled the savings vanish. Ro'zimatov (2026) report modest, tokenizer-dependent savings for six widely used tokenizers, using direct letter replacement, and call for pairing token counts with task evaluation. Neither measures LLM accuracy or compares apostrophe variants.

Uzbek evaluation. TUMLU (Isbarov et al., 2025) evaluates LLMs on Turkic languages including Uzbek in Latin and Cyrillic and finds small, inconsistent script gaps. UzLiB (Shopulatov, 2025) is an Uzbek linguistic benchmark. Sharipov et al. (2023) note that mixed apostrophe characters break Uzbek tokenization. The `uzbek2turkic` package (Sharipov, 2026) normalizes 1995 Uzbek Latin to the Common Turkic Alphabet, which is close to the 2026 alphabet but also maps *ng* to ñ; it uses suffix rules and an exceptions list. We did not evaluate it. Salaev et al. (2022) released a transliteration tool between Uzbek Cyrillic, the 1995 Latin alphabet and an earlier proposed new Latin alphabet; it predates the 2026 alphabet and we did not evaluate it either. Kletz et al. (2026) study script awareness of language models on multi-script languages such as Serbian and Kazakh, not Uzbek.

3 Data and Spelling Conditions

Sources. FLORES-200 `uzn_Latn` devtest (1,012 sentences) and dev (997) (NLLB Team et al., 2022) for RQ1; Belebele `uzn_Latn` (Bandarkar et al., 2024) (900 multiple-choice questions on 488 FLORES passages) for RQ2. Both are CC BY-SA 4.0 and written in 1995 Latin with the ASCII apostrophe (Belebele: 10,221 ASCII vs. 28 U+02BB). Gold answers are not balanced, so always answering the most common option scores 27.9%. For RQ5 we use 106,658 FineWeb-2 `uzn_Latn` documents (Hugging Face, 2024), converted the same way, with no FLORES documents; 1 of 29,072 Belebele 8-grams occurs in this corpus.

Conditions. We first canonicalize every apostrophe-like character to U+02BB after o/g and to U+02BC (tutuq belgisi) elsewhere. From this we derive: *okina* (official, canonical), *ascii*, *backtick* and *rsquo* (both marks replaced by one character, as a single keyboard key produces them); *new2026*; and *cyrillic*. For the ablation we build *only öğ* and *only şç*, which keep one half of the reform and undo the other. For RQ4 we add *back-converted* (new2026 converted back to 1995 spelling) and *mixed* (a seeded random apostrophe per word).

Conversion to the 2026 alphabet. Naive replacement wrongly converts foreign words (*show*→*şow*). We use YangiAlifbo[1], an open-source converter with a 959,740-form lexicon, a 400+ suffix morphological analyzer, a 112,447-word foreign-word filter and character-level CNNs, run in strict mode through a batch CLI that also splits hyphenated and slash-joined words. Output is used as-is, without manual correction. On FLORES devtest it converts 5,183 of 5,288 candidate words; misses are mostly Uzbek-spelled foreign names (*Polsha*, *Vashington*). The author, a native speaker, hand-checked a sample after a first review by an AI assistant, finding 13 of 709 candidate words missed (1.8%), errors in 7 of 100 FLORES sentences, and no false conversions.

Cyrillic is produced by a rule-based transliterator (digraphs, positional *e*→э/e, *-tsiya*→-ция); Latin-script names stay Latin. Some information is unrecoverable from 1995 Latin (soft sign, ц spelled *s*), so this condition is transliterated, not native Cyrillic. The same hand-check found errors in 41 of 2,413 words (1.7%), and in 19 of 100 FLORES sentences. All were in loanwords and names, such as ц written as с and missing soft signs.

4 Experimental Setup

Tokenizers (RQ1). o200k (GPT-4o/4.1/5), cl100k (GPT-4; identical to Phi-4), Llama 3.x, Qwen 2.5 (identical to Qwen 3), Gemma 3, Mistral Nemo (Tekken), DeepSeek V3, plus UTF-8 bytes. Fertility is tokens per word, with words counted once on the okina text. 95% CIs: 1,000 sentence-level bootstrap resamples.

Models (RQ2). Gemma 3 4B (bf16 and 4-bit), Qwen 2.5 7B, Llama 3.1 8B, Qwen 3 8B, Gemma 3 12B and Mistral Nemo 12B, all instruction-tuned; models above 4B run with 4-bit NF4 quantization (Dettmers et al., 2023) on one 12 GB GPU (RTX 4080 Laptop). Model revisions and package versions are logged.

Scoring. Zero-shot, with an English prompt frame that is byte-identical across conditions. The prediction is the option letter (A–D) with the highest next-token log-probability; no sampling. On this raw frame the four letters receive 61–94% of the probability mass depending on the model, so models do not always follow the format. We there-

[1] https://github.com/BilolbekYoldoshxojayev/yangicha-alifbo, commit `fdaafae`.

fore repeat okina and new2026 with each model's chat template and with an Uzbek frame written in the tested spelling.

Statistics. Paired comparisons against *okina*: exact McNemar tests, Holm-corrected over the nine comparisons per model, and 95% CIs from a cluster bootstrap over source articles with 1,000 resamples (questions sharing a passage are not independent).

Training (RQ5). Both experiments use the same documents and seeds for each spelling, three seeds (42–44), with the order of spellings alternated. Quality is validation bits per *word*, because word counts are equal across spellings while token and character counts are not. Evaluation time is excluded from training time. *Fine-tuning:* LoRA ($r = 16$) on 4-bit Gemma 3 4B, 1,133 documents (4M characters), 174 optimizer steps, then re-tested on Belebele. *From scratch:* a 98M-parameter GPT-2-style model (12 layers, dimension 768) on 53,241 documents (200M characters), one epoch, with a 16,000-entry byte-level BPE tokenizer that is either separate per spelling or shared (trained on both).

Validity checks. All 106 result files are complete and share one question order. Hashes of code and data did not change across the two restarts of the run. A repeated run of 100 predictions was identical, with zero log-probability difference. An early Llama run was discarded after a hash check found a corrupted weight file; 9–15% of its answers per condition differ from the repaired model, and all reported numbers come from the repaired model. In all training runs the GPU never reported thermal slowdown (maximum 66 °C).

5 Results

5.1 RQ1: Tokens

Figure 1 shows the change in tokens relative to *okina*. Relative to ASCII, the form most Uzbek text actually uses, the new alphabet changes token counts by between −3.0% (Llama 3) and +1.3% (DeepSeek V3), and by +1.1% on o200k. These agree in size with Ro'zimatov (2026) on a 60,390-word novel for Llama 3 (2.97% fewer) and Gemma (2.53% fewer, against our 2.0%). On o200k they report 0.10% fewer tokens against our 1.1% more; their own results for this tokenizer change sign between texts.

The official U+02BB is the *most* expensive Latin form for cl100k and Llama 3 (+5% over

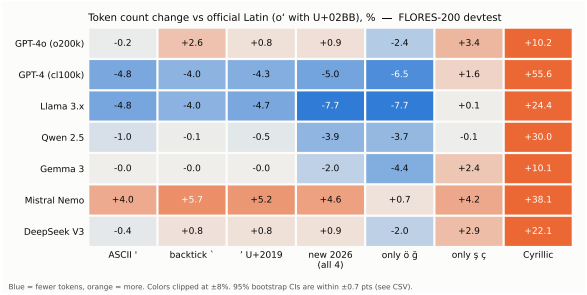


Figure 1: Token change (%) vs. official Latin on FLORES-200 devtest. Blue: fewer tokens; orange: more. All 95% CIs within ±0.7 points.

ASCII), but the cheapest for Mistral Nemo. Cyrillic costs +10% to +56%. The ablation splits the reform's effect: ö/ğ alone save 2.0–7.7% on six of seven tokenizers, while ş/ç alone change tokens by −0.1% to +4.2%, plausibly because *sh* and *ch* are frequent letter pairs that subword vocabularies already merge. Results replicate on FLORES dev within 0.3 points.

5.2 RQ2: Accuracy

Tables 1 and 2 and Figure 2 give the main result. The 2026 alphabet lowers accuracy in every model, by 3.1 to 9.6 points, significant after correction in all seven runs. Within the Gemma 3 family the effect shrinks with size (−7.4 at 4B, −3.1 at 12B), but size alone does not protect a model: Mistral Nemo, also 12B, has the largest drop (−9.6). The effect survives quantization (Gemma 3 4B: −6.8 at 4-bit).

None of the 28 apostrophe comparisons (three variants and mixed) is significant after correction. Cyrillic is significantly worse in four of seven runs, up to −18.6 (Mistral Nemo). In the ablation, neither half of the reform alone reproduces the full drop, and in 5 of 7 runs ş/ç cost more accuracy than ö/ğ, although ş/ç are the letters that do not save tokens. Token count and accuracy therefore do not move together.

Robustness. Table 3 repeats the comparison with the six 4-bit models under three frames. The gap is negative in all 18 cases. It is significant for five of six models with the chat template (−2.7 to −9.2; Qwen 2.5 is not significant) and five of six with the Uzbek frame (−1.8 to −9.8; Gemma 3 4B is not). The chat template raises letter mass to 95–100%, so the gap does not come from models ignoring the answer format. The Uzbek-frame result for Qwen 3 is weak, since only 27% of its probability mass falls on the letters.

Model	okina	ascii	backtick	’	mixed	new	öğ	şç	back-conv.	cyr
Gemma 3 4B bf16	64.0	−0.3	−0.3	−0.2	−0.1	**−7.4**	**−3.1**	**−5.9**	−0.1	**−4.7**
Gemma 3 4B 4-bit	59.9	+0.4	−0.2	+0.3	+0.8	**−6.8**	−1.3	**−3.9**	+0.0	−2.6
Qwen 2.5 7B	60.2	−0.6	−0.9	+0.2	−0.6	**−4.7**	−1.2	−0.4	+0.0	**−4.7**
Llama 3.1 8B	69.1	+0.6	+0.1	−0.1	+0.8	**−6.9**	−2.4	−3.0	+0.1	**−8.7**
Qwen 3 8B	69.7	+0.9	+0.4	+0.6	+1.2	**−4.3**	−1.8	−1.1	−0.1	−3.3
Gemma 3 12B	84.9	+1.0	+0.9	+0.8	+0.8	**−3.1**	−1.0	−1.4	+0.0	−0.7
Mistral Nemo 12B	74.7	+0.7	+0.3	−0.2	+0.2	**−9.6**	**−3.8**	**−6.9**	+0.0	**−18.6**

Table 1: Belebele accuracy (%) in official Latin, and change in points for every other spelling. Bold: significant after Holm correction over the nine comparisons per model (exact McNemar, $\alpha = 0.05$).

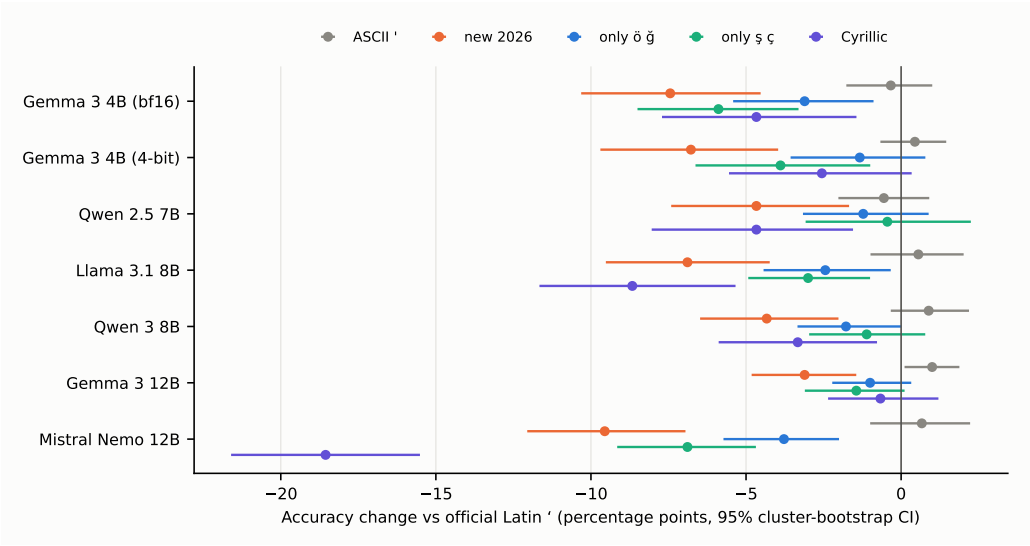


Figure 2: Accuracy change vs. official Latin, with 95% cluster-bootstrap CIs.

Model	new2026	Δ	95% CI	p
Gemma 3 4B bf16	56.6	−7.4	[−10.3, −4.5]	0.0000
Gemma 3 4B 4-bit	53.1	−6.8	[−9.7, −4.0]	0.0000
Qwen 2.5 7B	55.6	−4.7	[−7.4, −1.7]	0.0183
Llama 3.1 8B	62.2	−6.9	[−9.5, −4.2]	0.0000
Qwen 3 8B	65.3	−4.3	[−6.5, −2.0]	0.0021
Gemma 3 12B	81.8	−3.1	[−4.8, −1.4]	0.0035
Mistral Nemo 12B	65.1	−9.6	[−12.1, −7.0]	0.0000

Table 2: The new-alphabet gap: accuracy (%) on new2026, change from okina in points, 95% cluster-bootstrap CI, and Holm-corrected p.

Dose–response (exploratory). This analysis was chosen after seeing the data. Pooled over the six 4-bit models, the gap grows with the density of new letters (Table 4): −3.9, −6.1 and −7.7 points across thirds of questions, a slope of −0.98 points per additional new letter per 100 letters (95% CI [−1.76, −0.26]; −0.92 controlling for length). Five of six models have a negative slope, but only one has a CI excluding zero.

5.3 RQ3: Cost

Because API price is linear in tokens, the token changes above are cost changes. With prices checked on 30 Oct 2026, one million Uzbek words cost $3.28 (ASCII) vs. $3.31 (new alphabet) of GPT-5 input; Cyrillic costs $3.62. Table 5 shows usable context: relative to ASCII, the new alphabet moves it by about 3% at most, in either direction, while Cyrillic costs far more on some tokenizers.

5.4 RQ4: Normalization

Apostrophe variants, including random mixing, leave no significant gap to close. Back-converting new2026 removes the accuracy gap on our data: the change is at most 0.1 points in all seven runs, and answers match okina on at least 99.9% of questions in the six 4-bit models. The round trip is not perfect (99.69% of Belebele fields are identical; differences include all-caps forms such as *AQSH→AQSh*). This tests our own conversion, not text written natively in the new alphabet.

Model	English, raw			Chat template			Uzbek frame		
Gemma 3 4B	59.9	**−6.8**	94	64.8	**−5.8**	100	52.7	−1.8	70
Qwen 2.5 7B	60.2	**−4.7**	91	57.9	−2.7	100	60.0	**−3.8**	91
Llama 3.1 8B	69.1	**−6.9**	93	66.8	**−7.7**	100	65.8	**−9.8**	99
Qwen 3 8B	69.7	**−4.3**	72	69.8	**−4.4**	100	62.7	**−5.4**	27
Gemma 3 12B	84.9	**−3.1**	85	85.9	**−3.1**	100	83.2	**−3.3**	96
Mistral Nemo 12B	74.7	**−9.6**	61	72.2	**−9.2**	95	71.9	**−8.3**	74

Table 3: Robustness to the prompt frame, for okina and new2026. For each frame: okina accuracy (%), change on new2026 in points (bold: significant after Holm correction over six models), and mean probability mass on the four answer letters (%).

Model	Slope	95% CI	Low	Mid	High
Gemma 3 4B	−1.05	[−3.23, +0.74]	−4.0	−7.0	−9.3
Qwen 2.5 7B	−1.24	[−3.21, +0.64]	−4.0	−2.0	−8.0
Llama 3.1 8B	−1.22	[−3.23, +0.74]	−2.3	−8.7	−9.7
Qwen 3 8B	+0.28	[−1.17, +1.78]	−6.0	−3.3	−3.7
Gemma 3 12B	−1.15	[−2.38, −0.11]	−1.7	−3.0	−4.7
Mistral Nemo 12B	−1.49	[−3.20, +0.23]	−5.7	−12.3	−10.7
Pooled, 6 models	−0.98	[−1.76, −0.26]	−3.9	−6.1	−7.7

Table 4: Dose–response (exploratory). Slope: points of accuracy gap per additional new letter per 100 letters, with 95% cluster-bootstrap CI. Thirds: gap in points, from fewest to most new letters.

Tokenizer	okina	ascii	new	cyr
DeepSeek V3	40,300	40,500	40,000	33,000
GPT-4	37,700	39,600	39,700	24,200
GPT-4o	48,700	48,900	48,300	44,200
Gemma 3	45,100	45,100	46,000	41,000
Llama 3.x	38,500	40,500	41,700	31,000
Mistral Nemo	47,900	46,000	45,800	34,700
Qwen 2.5	38,900	39,300	40,500	29,900

Table 5: Uzbek words that fit in a 128,000-token context, by tokenizer (FLORES devtest).

Seed	Trained on old				Trained on new			
	min	tok/s	old	new	min	tok/s	old	new
42	63.0	378	57.9	54.7	56.4	416	58.6	58.6
43	63.0	378	58.8	54.3	57.7	407	57.6	57.8
44	55.9	426	59.4	55.1	55.1	426	57.6	58.3

Table 6: LoRA runs per seed: training minutes, tokens/s, and accuracy (%) on old and new text, trained on old or new spelling. Base model: 59.9 on old text, 53.1 on new.

Tokenizer	Seed	Min		Wh		Bits/word	
		old	new	old	new	old	new
separate	42	38.3	28.0	58.8	52.2	12.05	11.97
separate	43	27.9	28.0	52.1	52.3	12.31	12.04
separate	44	27.8	28.0	52.0	52.4	11.98	12.01
shared	42	28.6	28.6	53.4	53.4	11.87	12.04
shared	43	28.6	28.6	53.4	53.4	11.86	12.25
shared	44	28.6	28.6	53.4	53.5	12.16	11.96

Table 7: From-scratch runs per seed: training minutes, GPU energy (Wh) and final validation bits per word, on old and new spelling.

5.5 RQ5: Training

Table 8 summarizes cost and quality, and Tables 6 and 7 give every run.

Cost. Token counts differ little: −1.5% for Gemma's tokenizer with LoRA, and +0.27% (separate) and +0.08% (shared) from scratch, even though the new-spelling text has fewer characters. The new alphabet therefore brings no meaningful saving in our trained tokenizers either. Peak memory is identical (6.58 and 9.52 GB).

Timing differences reflect machine state more than spelling. One from-scratch old-spelling run was 27% slower, with lower GPU power and no thermal flag (cause unknown); without it, old and new take 27.9 and 28.0 minutes. LoRA throughput varied within spelling (378–426 tokens/s), and the one pair with equal throughput differs by 1.5%, matching the token count.

Quality from scratch. Learning curves nearly overlap (Figure 3). The bits-per-word differences change sign across seeds and tokenizer modes and are within seed-to-seed spread, so we detect no quality difference with three seeds.

Fine-tuning and accuracy. Fine-tuning changes the accuracy gap. Mean over three seeds, on Gemma 3 4B (4-bit), the new-minus-old-text

Setup	Sp.	Tokens (M)	Min	Wh	Bits/word
LoRA	old	1.43	60.6 ± 4.1	75.8	9.47 ± 0.01
	new	1.41	56.4 ± 1.3	73.4	10.01 ± 0.00
Scratch, sep.	old	43.73	31.3 ± 6.0	54.3	12.11 ± 0.17
	new	43.85	28.0 ± 0.0	52.3	12.01 ± 0.04
Scratch, shared	old	44.74	28.6 ± 0.0	53.4	11.96 ± 0.17
	new	44.77	28.6 ± 0.0	53.4	12.09 ± 0.15

Table 8: Training on old vs. new spelling: mean ± SD over three seeds. Bits per word on validation text in the same spelling.

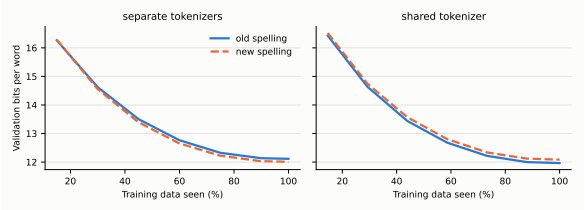


Figure 3: From-scratch learning curves, mean of three seeds.

gap is -6.8 points $[-9.7, -4.0]$ for the base model, -4.0 $[-6.1, -1.9]$ after training on old spelling, and $+0.3$ $[-1.1, +1.8]$ after training on new spelling. On new-alphabet questions, the new-trained model beats the base by 5.1 points $[+2.6, +7.9]$ and the old-trained model by 3.5 $[+1.5, +5.6]$, significant in each seed after Holm correction over the nine per-seed comparisons. Accuracy of each model on its own spelling does not differ detectably (-0.5 $[-2.1, +1.2]$), and training on new spelling does not significantly hurt old-spelling accuracy compared with the base (-2.0 $[-4.3, +0.4]$). LoRA did not raise accuracy on old text above the base model, so this result concerns the gap between spellings, not overall skill. After training, the model still predicts new-spelling text less well (10.01 vs. 9.47 bits per word).

6 Discussion

The reform's token savings are small and tokenizer-dependent, consistent with Ibadullaev et al. (2026) and Ro'zimatov (2026). Its accuracy cost, by contrast, is consistent across models and prompt frames, and it does not track tokens: the letters that save tokens are not the ones that cost the most accuracy.

Our training results point to distribution shift: current models saw little Uzbek written with ö, ğ, ş, ç, and a small amount of such text closes the gap in the one model we tested. The cost therefore looks like a *transition* cost that should shrink as new-alphabet text enters training data. From scratch, neither spelling has a clear advantage.

These results speak to the public dispute only in part. We find no evidence that the new letters make models cheaper to train or harder to teach. In the short term the reform does cost accuracy on current models, which supports neither the view that the letters make no difference to AI nor a claim of immediate technical gain.

For practitioners during the 2027–2031 transition, back-converting new-alphabet input to the 1995 spelling is a cheap fix for current models, and standardizing apostrophes is, on present evidence, not needed for accuracy. Model developers can likely reduce the gap by including new-alphabet text in training data.

Limitations

New-alphabet and Cyrillic text are converted, not written by people; converter errors and unrecoverable Cyrillic distinctions may affect results, and our error rates come from one annotator, the author of the converter, working after a first AI review, so they are not independent and some errors may have been missed. Because misses leave old-spelling words in place, they may understate the gap slightly. Fine-tuning used the same converter as the test text, so the model may partly be learning that converter's output, and it was tested on one model only. Three seeds detect only large training effects, and our CIs for averaged seeds are not corrected for multiple comparisons. The dose–response analysis is exploratory.

FLORES/Belebele are translated encyclopedic and news text. We test one task, zero-shot. We had limited GPU power, one 12 GB laptop GPU, so we test only open models up to 12B, mostly 4-bit quantized, fine-tune on 4M characters and train only a 98M-parameter model from scratch. Proprietary models (Claude, Gemini, GPT) are not evaluated. Mirrors of gated repositories were used for some models. Accuracy runs started before the laptop's cooling settings were fixed, but only training timings depend on that, and those came after.

Data and Code

Code, the parallel data, per-question results and logs are available at `https://github.com/BilolbekYoldoshxojayev/uzbek-alphabet-llm`. Model weights and the training corpus are not included; the corpus can

be rebuilt with the provided script.

Ethics and Use of AI Assistants

All benchmark data is public under CC BY-SA 4.0. The author conceived the study, defined its research questions, spelling conditions and rigor requirements, proposed the old-versus-new training comparison, built the YangiAlifbo converter, chose the benchmark, hand-checked the converted text samples, and reviewed the final paper. An AI assistant (Claude) wrote the evaluation and analysis code, ran the experiments, did a first review of the converted text samples, contributed to the detailed study design, and drafted the text. The author takes responsibility for the content.

References

Lucas Bandarkar, Davis Liang, Benjamin Muller, Mikel Artetxe, Satya Narayan Shukla, Donald Husa, Naman Goyal, Abhinandan Krishnan, Luke Zettlemoyer, and Madian Khabsa. 2024. The Belebele benchmark: a parallel reading comprehension dataset in 122 language variants. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 749–775.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*.

Gazeta.uz. 2026. Prezident podpisal zakon o novom variante uzbekskogo alfavita v den' uzbekskogo yazyka. 21 October. https://www.gazeta.uz/ru/2026/10/21/uzbek-alphabet/.

Hugging Face. 2024. FineWeb-2. https://huggingface.co/datasets/HuggingFaceFW/fineweb-2, revision af9c133.

Doniyor Ibadullaev, Mohira Hayitova, Nodir Gafurov, Zarifa Salmanova, and Munisa Izrailbekova. 2026. Uzbek orthography reform and subword tokenization: Code and results. Zenodo, doi:10.5281/zenodo.23665432. Code for "Orthography or tokenizer?", submitted to PeerJ Computer Science.

Jafar Isbarov, Arofat Akhundjanova, Mammad Hajili, Kavsar Huseynova, Dmitry Gaynullin, Anar Rzayev, Osman Tursun, Aizirek Turdubaeva, Ilshat Saetov, Rinat Kharisov, Saule Belginova, Ariana Kenbayeva, Amina Alisheva, Abdullatif Köksal, Samir Rustamov, and Duygu Ataman. 2025. TUMLU: A unified and native language understanding benchmark for Turkic languages. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*. ArXiv:2502.11020.

David Kletz, Sandra Mitrović, Ljiljana Dolamić, and Fabio Rinaldi. 2026. Are language models script-aware? arXiv:2610.08037. Accepted to AACL-IJCNLP 2026.

Legal Alliance. 2026. Senate approves a new 28-letter Uzbek alphabet. 10 September. https://legalalliance.uz/en/news/novyy-uzbekskiy-alfavit-28-bukv-senat.

NLLB Team, Marta R. Costa-jussà, et al. 2022. No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*.

Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. 2023. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems*.

Xayrullo Ro'zimatov. 2026. The tokenization premium: Measuring Uzbek tokenization overhead in large language models and evaluating the 2026 alphabet reform. Zenodo, record 21872526. Also listed in BG Pulse Journal USA 2(08):340–352.

Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. 2021. How good is your tokenizer? on the monolingual performance of multilingual language models. In *Proceedings of ACL-IJCNLP*.

Ulugbek Salaev, Elmurod Kuriyozov, and Carlos G'omez-Rodr'iguez. 2022. A machine transliteration tool between Uzbek alphabets. In *Proceedings of the International Conference on Agglutinative Language Technologies as a Challenge of Natural Language Processing (ALTNLP)*. ArXiv:2205.09578.

Maksud Sharipov. 2026. uzbek2turkic: A normalizer from Uzbek Latin to the Common Turkic Alphabet. Python package, version 0.2.2. https://pypi.org/project/uzbek2turkic/.

Maksud Sharipov, Elmurod Kuriyozov, Ollabergan Yuldashev, and Ogabek Sobirov. 2023. UzbekTagger: The rule-based POS tagger for Uzbek language. arXiv:2301.12711. Accepted to the 10th Language & Technology Conference, Poznan.

Abror Shopulatov. 2025. UzLiB: A benchmark for evaluating LLMs on Uzbek linguistics. https://huggingface.co/datasets/tahrirchi/uzlib.

Ilhomjon Sobirov. 2026. Yangi alifboga o'tishga siyosiy iroda yetadimi? Anhor.uz, 23 October. https://anhor.uz/uzl/ichkaridan-nazar/yangi-alifbo-siyosiy-iroda.

Anna Wegmann, Dong Nguyen, and David Jurgens. 2025. Tokenization is sensitive to language variation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 10958–10983.